HJudge

Đặng Xuân Minh Hiếu

2026-03-15

Uriel

Outline

Introduction 2
Architecture Overview 9
Components 14
Process 17
Testing 21
Roadmap 24

Introduction

As a coach, I want to train my trainees by creating weekly problemsets, and monitor their progress.

Context

Nowadays, a lot of Online Judges, e.g. Codeforces, AtCoder

- each has its strength

Context

Nowadays, a lot of Online Judges, e.g. Codeforces, AtCoder

- each has its strength

Creating a decent problemset

$\Leftrightarrow$ combining problems from different OJs

Context

Nowadays, a lot of Online Judges, e.g. Codeforces, AtCoder

- each has its strength

Creating a decent problemset

$\Leftrightarrow$ combining problems from different OJs

$\Leftrightarrow$ **difficult to monitor**

Context

Nowadays, a lot of Online Judges, e.g. Codeforces, AtCoder

- each has its strength

Creating a decent problemset

$\Leftrightarrow$ combining problems from different OJs

$\Leftrightarrow$ **difficult to monitor**

$\Rightarrow$ need a tool to integrate all OJ

Context

Current solutions:

	USACO Guide's Group	**VJudge**
Support	All popular judges	All popular judges
Lesson management	✓	✓
Submit	∅	✓
See submissions	∅	∅

Problem definition

Main problem:

- System can import *any* **exercise** from *any* *supported* **Judge**.
- System can crawl *all* **submissions** of *all* **users** related to the *imported* **exercise**
  - **User** will provide its **handles** (a.k.a username) of each respective **Judge**.

Problem definition

Main problem:

- System can import *any* **exercise** from *any* *supported* **Judge**.
- System can crawl *all* **submissions** of *all* **users** related to the *imported* **exercise**
  - **User** will provide its **handles** (a.k.a username) of each respective **Judge**.

Side problem:

- **User** can create **Course**, and become **CourseAdmin**
- **CourseAdmin** can create **Lesson** in a **Course**
- **CourseAdmin** can import **Exercise** to a **Lesson**
- There exists a mechanism to track the **progress** of the **participants**

Problem definition – Assumption

1. The existence of an exercise or a submission is eternity, i.e. it will never be deleted.
2. The content of a submission is never changed.
3. The title of a problem is never changed.

Problem definition – Assumption

1. The existence of an exercise or a submission is eternity, i.e. it will never be deleted.
2. The content of a submission is never changed.
3. The title of a problem is never changed.

(temporarily)

4. **User** never change their **handle**

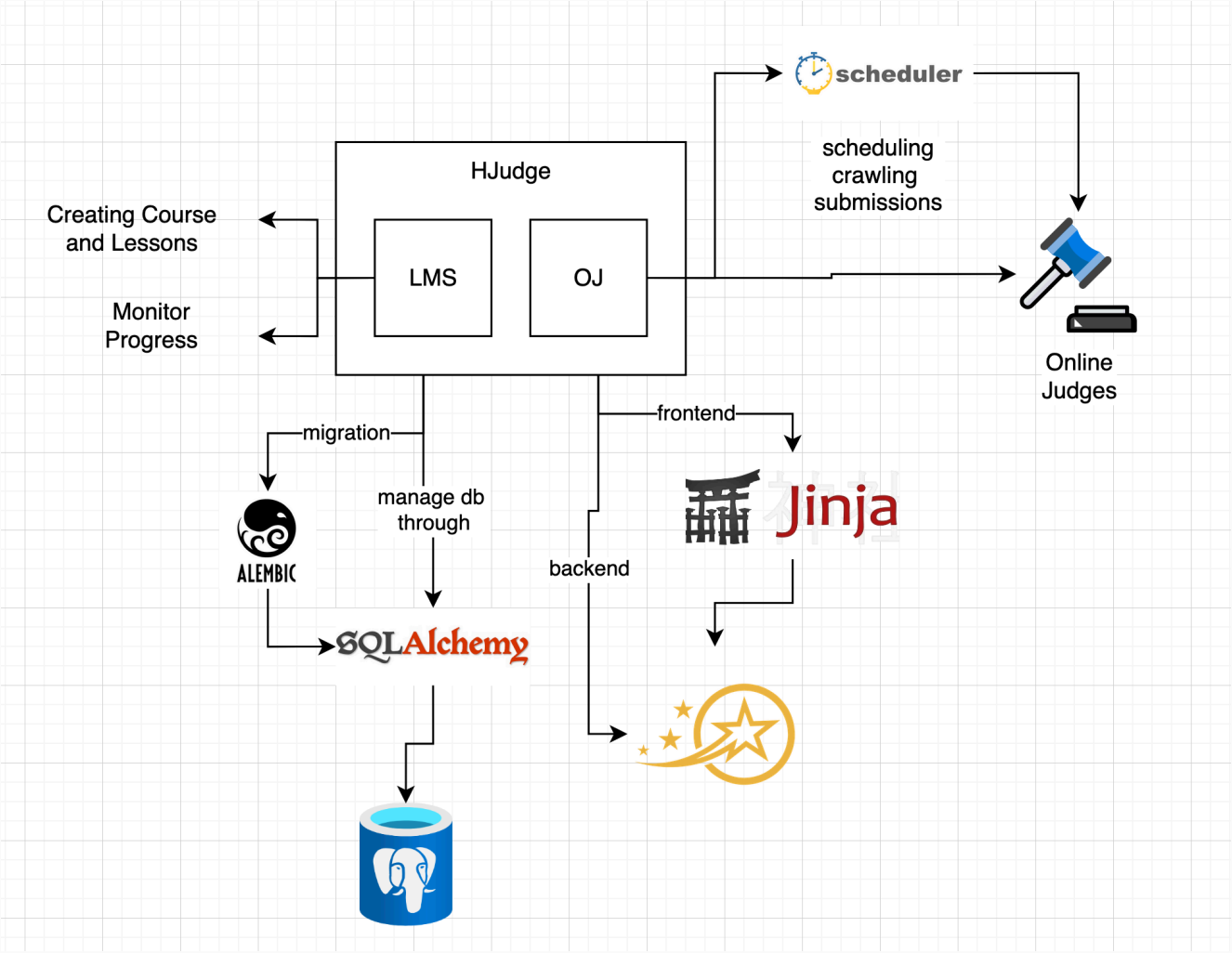
Problem definition – Glossary

1. **Judge**(s): a system that contains exercises to solve, and could execute and evaluate programs from the user that attempt solve an exercise.
2. **Exercise**: a problem of a Judge[1]
3. **Submission**: a program in the form of code, that the user submits, and evaluated by the Judge
4. **Verdict** of a Submission:
   - Accepted
   - Rejected, because of: Time-Limit-Exceed, Wrong-Answer, Runtime-Error, Compile-Error, etc.

[1] I decided to use the word "Exercise" instead of "Problem"

Architecture Overview

Summary


scheduler
scheduling
crawling
submissions
HJudge
Creating Course
and Lessons
LMS
OJ
Monitor
Progress
Online
Judges
frontend
migration
manage db
through
Jinja
ALEMBIC
backend
SQLAlchemy

Summary

Others:

- Testing: Pytest + SQLite
- Package manager: uv

Constraints and Decisions

1. Already-made decisions: Use the tech-stack currently use in Uriel, if known
   - Litestar for backend
   - DB: PostgreSQL + SQLAlchemy
     - Alembic for migration, not sure how Uriel is doing this
   - uv for package management and venv
   - Pytest + coverage for testing

Constraints and Decisions

1. Already-made decisions: Use the tech-stack currently use in Uriel, if known
   - Litestar for backend
   - DB: PostgreSQL + SQLAlchemy
     - Alembic for migration, not sure how Uriel is doing this
   - uv for package management and venv
   - Pytest + coverage for testing
2. Considering: how to fast-deliver?
   - Front-end: Jinja
   - Scheduler: Claude gave me solutions, I picked one: APScheduler

Constraints and Decisions

3. (goal) Split the LMS module and OJ module
   - “Uriel mình microservice till die”
   - can not deliver this yet due to time constraint
   - still, try to develop the two modules as independent as possible

Components

OJService

Responsibility:

- Handle all tasks relating to integration with various Online Judges

Features:

- Crawl the content of an exercise
- Schedulely crawl latest submissions of users in supported Judges
- Given an exercise and a user, return respective submissions and URLs

LMService

Responsibility and Features:

- Crawl the content of an exercise
- Schedulely crawl submissions of users in supported Judges, and linked to its urls

Features

- Creating and updating courses and lessons
- Monitor the progress of participants

Process

Pattern used

- Abstract Factory:
  - given a DB service, yield a Unit-of-Work which is a Repository factory
  - given the crawler service, yield a Judge factory
- Singleton:
  - for each UoW, only one instance per type of Repository are created.

How I created a functionality

1. Define the model class
   - defined the functionality
   - necessary fields and methods

How I created a functionality

1. Define the model class
   - defined the functionality
   - necessary fields and methods
2. Define the necessary entity of the respective class
   - cannot make model as entity, as after `session.commit()`, data is inaccessible because they are expired.

How I created a functionality

1. Define the model class
   - defined the functionality
   - necessary fields and methods
2. Define the necessary entity of the respective class
   - cannot make model as entity, as after `session.commit()`, data is inaccessible because they are expired.
3. Define the Repository
   - basic CRUD
   - separate the logic of DB from services

How I created a functionality

Each functionality corresponds to one endpoint, which corresponds to one function service.

How I created a functionality

Each functionality corresponds to one endpoint, which corresponds to one function service.

4. Define the service functions
   - The params are inject with an `AbstractUnitOfWork` and sometimes `JudgeFactory`

How I created a functionality

Each functionality corresponds to one endpoint, which corresponds to one function service.

4. Define the service functions
   - The params are inject with an `AbstractUnitOfWork` and sometimes `JudgeFactory`
5. Define the endpoints
   - Litestar backend is inject with an `AbstractUOWFactory` and `JudgeFactory` (as which judge to choose is decided in the service layer)

Testing

Process of test

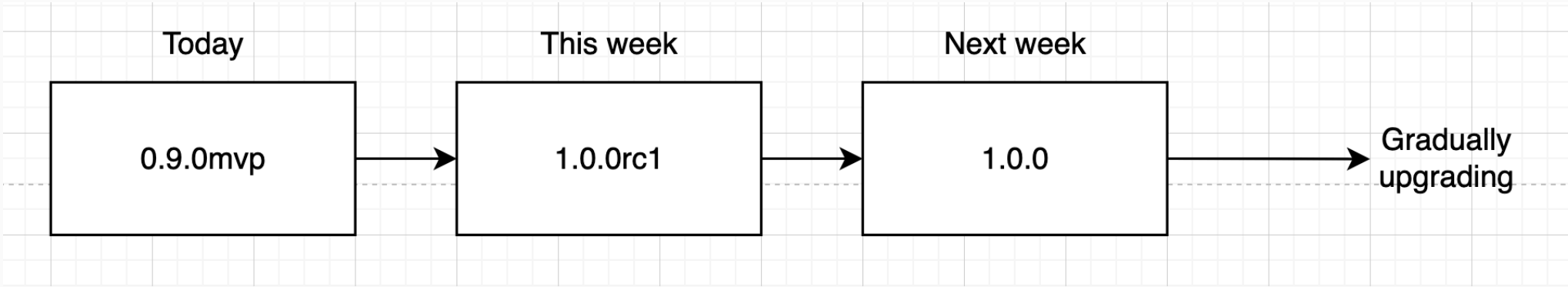
- This project tries to follow test-driven development:
  - write the test first,
  - implementation follows.
- Most of the test is in the service layer
- Test purely in local: using mock to mimic actual Judge's response
- Aim: cover at least 80% of the source code

Stress test

- will try to implement, aim: 100+ user

Roadmap

Release plan



- 0.9.0mvp: with darkmode
  - support Codeforces only
  - enable adding course and lessons
  - user can see their submission on Judges updated on HJudge
- 1.0.0rc1:
  - progress and dashboard of lessons/courses
- 1.0.0: officially release
  - support more judges (AtCoder, LQDOJ)

Future features

- Support more judges
- Support viewing submissions (rendering on client side)
- Adding Markdown support $\Rightarrow$ lecture resources
- Improve UX